

ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES
De Béni Mellal

TP 2

Monolithe Modulaire avec Spring Boot

Restructuration et Préparation à la Migration

Réalisé par :

BAH Thierno Mamoudou

Encadrant :

Pr. BE ELBAGHAZAOUI

Année universitaire : 2025 — 2026

1. Introduction et Objectifs

Ce deuxième travail pratique s'inscrit dans la continuité du TP1 en poussant plus loin la qualité architecturale de l'application Spring Boot. Le monolithe simple développé précédemment est ici restructuré en un monolithe modulaire, où chaque domaine métier (Produit, Client, Commande) est isolé dans son propre module avec des frontières claires et une communication inter-modules définie.

Cette démarche est fondamentale : elle prépare le terrain pour une future migration vers une architecture microservices, tout en maintenant la simplicité d'un déploiement unique. Le patron de conception DTO (Data Transfer Object) est introduit pour découpler la représentation interne des données de leur exposition via l'API REST.

Objectifs du TP

- Restructurer le monolithe du TP1 en modules métier distincts (product, customer, order).
- Introduire le pattern DTO pour séparer les entités JPA des objets exposés à l'API.
- Utiliser MapStruct pour automatiser la conversion Entity ↔ DTO.
- Séparer l'interface de service de son implémentation (ProductService / ProductServiceImpl).
- Implémenter les modules Customer et Order complets avec leurs couches respectives.
- Créer une méthode existsById() dans CustomerService pour vérifier l'existence d'un client.
- Développer un endpoint pour consulter l'historique des commandes d'un client.

Comparaison TP1 vs TP2

Aspect	TP1 — Monolithe Simple	TP2 — Monolithe Modulaire
Couches	Entity, Repository, Service, Controller	+ DTO, Mapper, Service Interface
Exposition API	Entité JPA directement	DTO uniquement
Mapping	Manuel ou absent	MapStruct automatique
Service	Classe concrète unique	Interface + Implémentation séparées
Modules	Product uniquement	Product + Customer + Order
Communication	Accès direct au repository	Via services publics uniquement

2. Architecture Modulaire du Projet

L'application est organisée en trois modules métier autonomes. Chaque module possède exactement la même structure interne en couches, garantissant cohérence et maintenabilité. Le module Order est le seul à dépendre des autres modules, uniquement via leurs interfaces de service publiques — jamais via leurs repositories.

Diagramme des dépendances inter-modules

Comme illustré dans le TP, les dépendances entre modules respectent une règle stricte : le module Order consomme uniquement les services publics de Product et Customer, sans jamais accéder à leurs repositories. Product et Customer sont totalement indépendants l'un de l'autre.

Module	Dépendances	Règle
Product Module	Aucune	Totalement autonome
Customer Module	Aucune	Totalement autonome
Order Module	ProductService, CustomerService	Via interfaces uniquement, jamais les repositories

3. Module Product — Refactoring avec DTO et MapStruct

Le module Product du TP1 est entièrement refactorisé. L'entité Product n'est plus exposée directement dans le contrôleur : elle est remplacée par des objets DTO dédiés. MapStruct génère automatiquement le code de conversion entre ces deux représentations à la compilation.

3.1 Vérification préliminaire — GET /api/products

Avant de refactoriser, on vérifie que les données héritées du TP1 sont toujours présentes. Un GET sur /api/products retourne la liste complète des produits avec le code 200 OK. On constate les produits créés lors du TP1 : PC Gamer Zen, Souris Ergonomique, Clavier Mécanique, etc.

GET `{{base_url}} /api/products`

Docs Params Authorization Headers (6) Body Scripts ● Settings

Query Params

	Key	Value
--	-----	-------

Body Cookies Headers (5) Test Results (1/1) 200 OK • 195

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
2 {
3   "id": 2,
4   "name": "Telephone",
5   "description": "Telephone de grande valeur",
6   "price": 3200.00,
7   "stock": 10,
8   "category": null
9 },
10 {
11   "id": 6,
12   "name": "Refrigerateur",
13   "description": "Taille moyenne",
14   "price": 7500.00,
15   "stock": 5,
16   "category": {
```

Figure 1 — GET /api/products : liste des produits (200 OK) — héritage du TP1

3.2 POST /api/products — Création via DTO

Après refactoring, la création d'un produit passe par le `CreateProductRequest`. Ici on crée "PC" à 12000,00 avec un stock de 3. La réponse est un `ProductDTO` (200 Created) — l'entité JPA n'est plus jamais exposée. Donc pas de stock à la sortie.

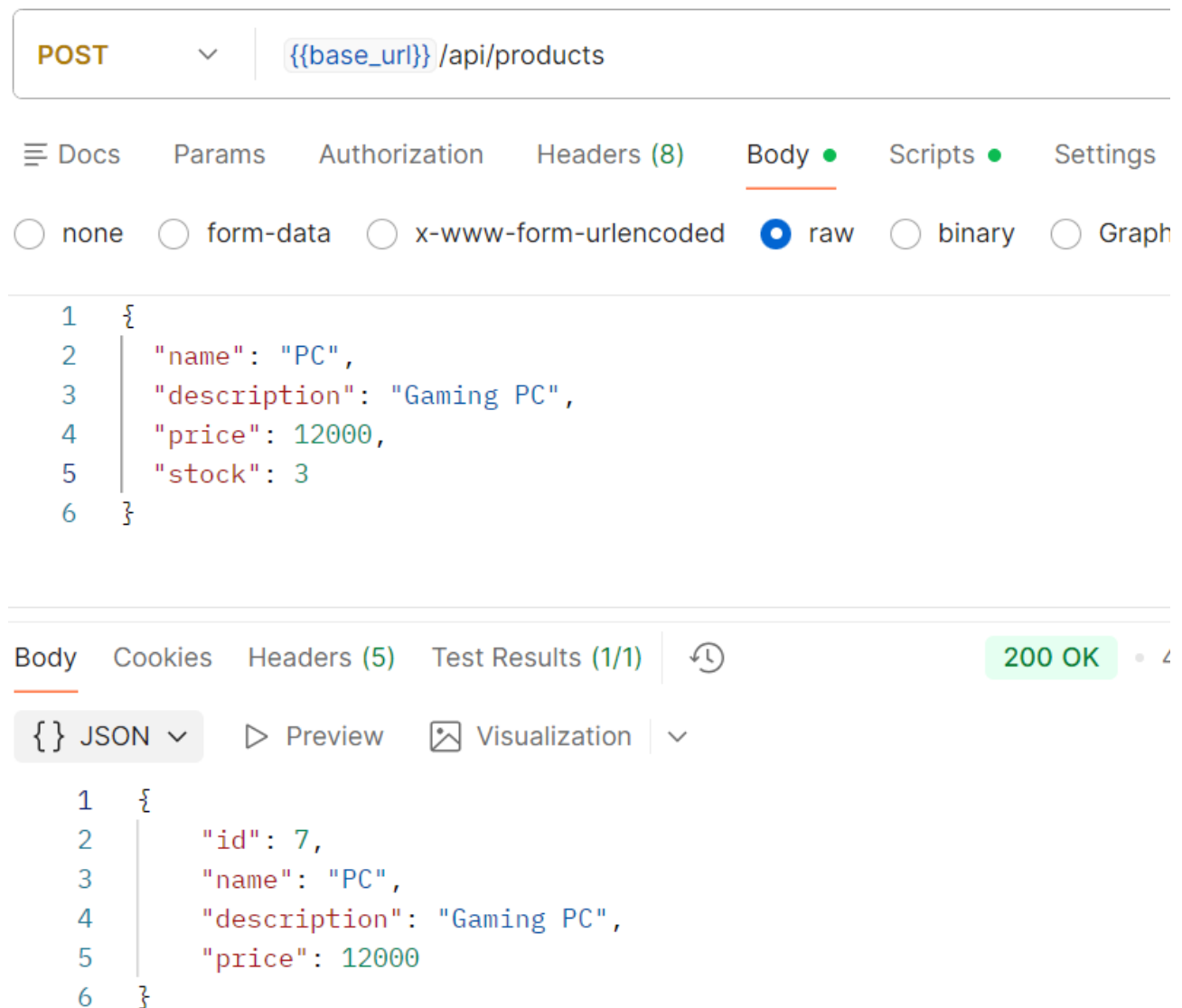


Figure 2 — POST /api/products : création via `CreateProductRequest`, réponse `ProductDTO` (201 Created)

4. Tests Postman — Module Customer

Le module Customer est testé via Postman. Deux clients sont créés pour valider le bon fonctionnement des endpoints, puis la liste est consultée pour vérifier la persistance des données.

4.1 POST /api/customers — Création du client 1

Le premier client créé est Mahmud (email : `tmb@gmail.com`). Le corps de la requête est un objet JSON conforme au `CustomerDTO`. La réponse 200 Created renvoie le `CustomerDTO` avec l'id généré (1) et tous les champs correctement persistés.

The screenshot displays a REST client interface for a POST request to `{{base_url}}/api/customers`. The request body is a JSON object: `{ "name": "Mahmud", "email": "tmb@gmail.com" }`. The response is a 200 OK status with a body containing: `{ "id": 2, "name": "Mahmud", "email": "tmb@gmail.com" }`. The interface includes tabs for Docs, Params, Authorization, Headers (8), Body (selected), Scripts, and Settings. Below the tabs are radio buttons for content types: none, form-data, x-www-form-urlencoded, raw (selected), binary, and GraphQL. The response status is 200 OK, and the body is shown in JSON format.

```
1 {
2   "name": "Mahmud",
3   "email": "tmb@gmail.com"
4 }
```

Body Cookies Headers (5) Test Results (1/1) 200 OK • 52

{ } JSON Preview Visualization

```
1 {
2   "id": 2,
3   "name": "Mahmud",
4   "email": "tmb@gmail.com"
5 }
```

Figure 16 — POST `/api/customers` : création de Mahmud (201 Created, id=2)

4.2 POST `/api/customers` — Création du client 2

Le deuxième client créé est Lamine (email : `lmn@gmail.com`). La réponse 200 Created confirme la création avec l'id 2, démontrant l'auto-incrémentation correcte de la clé primaire.

The screenshot displays a REST client interface. At the top, the method is set to **POST** and the URL is `{{base_url}}/api/customers`. Below the URL bar, there are tabs for **Docs**, **Params**, **Authorization**, **Headers (8)**, **Body** (selected), **Scripts**, and **Settings**. Under the **Body** tab, the content type is set to **raw** (selected), with other options being **none**, **form-data**, **x-www-form-urlencoded**, **binary**, and **GraphQL**. The request body is a JSON object:

```
1 {
2   "name": "Lamine",
3   "email": "lmn@gmail.com"
4 }
```

 Below the request body, there are tabs for **Body** (selected), **Cookies**, **Headers (5)**, and **Test Results (1/1)**. The **Test Results** tab shows a status of **200 OK**. Below the tabs, there are options for **JSON** (selected), **Preview**, and **Visualization**. The response body is a JSON object:

```
1 {
2   "id": 3,
3   "name": "Lamine",
4   "email": "lmn@gmail.com"
5 }
```

Figure 17 — POST /api/customers : création de Lamine (200 Created, id=3)

4.3 GET /api/customers — Liste des clients

Un GET sur /api/customers confirme la présence des deux clients en base. La réponse 200 OK retourne un tableau JSON avec les deux CustomerDTO. La structure de la réponse respecte bien le DTO défini : id, firstName, lastName, email.

GET
{{base_url}}/api/customers

Docs
Params
Authorization
Headers (6)
Body
Scripts
Settings

Query Params

	Key	Value	Descrip
Body			

200 OK • 223 ms • 31

JSON
Preview
Visualize

```

1  [
2    {
3      "id": 1,
4      "name": "Moudma",
5      "email": "moudma@test.com"
6    },
7    {
8      "id": 2,
9      "name": "Mahmud",
10     "email": "tmb@gmail.com"
11   },
12   {
13     "id": 3,
14     "name": "Lamine",
15     "email": "lmn@gmail.com"
16   }
17 ]

```

Figure 18 — GET /api/customers : liste des 2 clients (200 OK)

5. Tests Postman — Module Order

Le module Order est testé en créant une commande liée au client d'ID 1 (Thierno Hachimi), puis en consultant la liste de toutes les commandes.

5.1 POST /api/orders — Création d'une commande

Une commande est créée. La réponse 200 Created retourne l'OrderDTO complet avec l'id généré (2), la date de commande générée automatiquement au moment de la persistance.

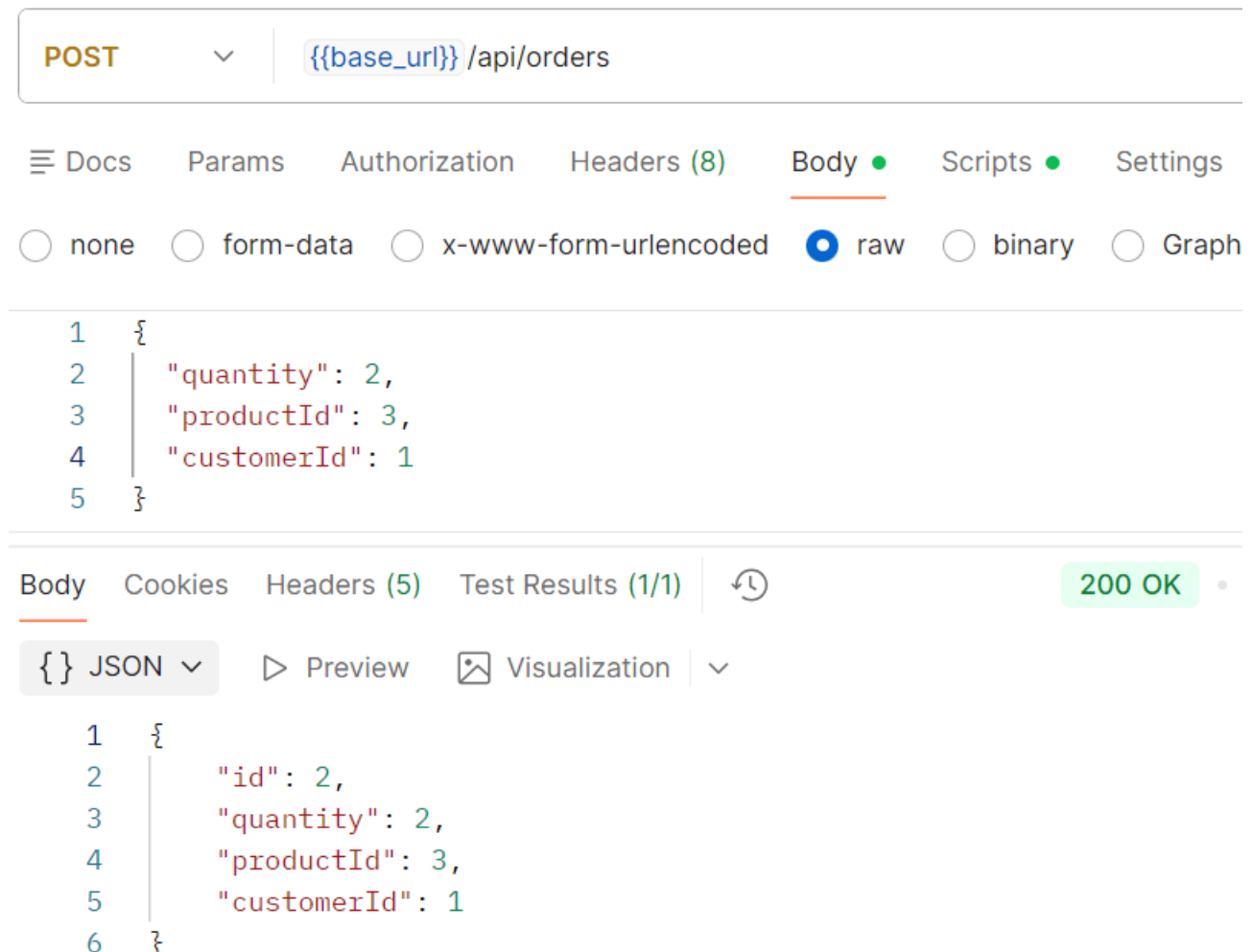


Figure 25 — POST /api/orders

5.2 GET /api/orders — Liste de toutes les commandes

Un GET sur `/api/orders` retourne la liste de toutes les commandes. La réponse 200 OK confirme la présence de la commande créée avec tous ses champs : `id`, `orderDate`, `totalAmount`, `status` et `customerId`. Ce dernier champ illustre le choix architectural de l'OrderDTO qui n'expose que l'identifiant du client.

GET

▼

{{base_url}}/api/orders

☰ Docs

Params

Authorization

Headers (6)

Body

Scripts ●

Settings

Query Params

	Key	Value	De
--	-----	-------	----

Body

Cookies

Headers (5)

Test Results (1/1)

🔄

200 OK • 23 ms

{{}} JSON ▼

▶ Preview

🖼️ Visualize ▼

```
1  [  
2      {  
3          "id": 1,  
4          "quantity": 2,  
5          "productId": 2,  
6          "customerId": 1  
7      },  
8      {  
9          "id": 2,  
10         "quantity": 2,  
11         "productId": 3,  
12         "customerId": 1  
13     }  
14 ]
```

Figure 26 — GET /api/orders : liste des commandes (200 OK) avec customerId visible

6. Conclusion et Bilan

Ce TP2 a permis de transformer une application monolithique simple en une architecture modulaire bien structurée, prête pour une éventuelle migration vers les microservices. Chaque module est désormais autonome, possède ses propres couches et communique avec les autres uniquement via des interfaces de service.

Compétences acquises

- Conception du pattern DTO pour découpler l'API de la base de données.
- Utilisation de MapStruct pour générer automatiquement le code de mapping à la compilation.
- Séparation service interface / service implémentation pour un code plus maintenable et testable.
- Modélisation d'une architecture modulaire avec des frontières de domaine claires.
- Implémentation d'une relation @ManyToOne entre Order et Customer avec gestion dans le mapper.
- Utilisation du mapper inter-couches pour éviter l'exposition directe des entités JPA.
- Validation de l'existence d'un client avant création d'une commande (existsById).

Avantages du Monolithe Modulaire vs Microservices

Critère	Monolithe Modulaire	Microservices
Déploiement	Simple — un seul artefact	Complexe — N services
Communication	Appels de méthodes Java	HTTP / Messaging (Kafka...)
Transaction	ACID native JPA	Saga pattern nécessaire
Frontières domaines	Clares — interfaces de service	Enforced — réseau
Scalabilité	Limitée — tout ou rien	Granulaire par service
Migration future	Facile — modules isolés	Déjà en microservices

Le monolithe modulaire représente une excellente étape intermédiaire entre le monolithe simple du TP1 et une architecture microservices complète. Il offre la rigueur architecturale des microservices (modules indépendants, interfaces claires, DTOs) tout en conservant la simplicité opérationnelle d'un déploiement unique. Cette approche est idéale pour des équipes en croissance ou des applications de taille moyenne.